# Assignment 3: Understanding Algorithm Efficiency and Scalability

Saketh Chowdary Nalluri

Algorithms and Data Structures

07 June, 2026

## Introduction

In this assignment, the efficiency and scalability of two important data structure and algorithm concepts: Randomized Quicksort and Hashing with Chaining will be examined. The aim is to understand their theoretical performance, code them in Python, and test their behavior with varying inputs.

## Part - 1 (Randomized Quicksort)

### Implementation

Randomized Quicksort is one of the variants of Quicksort algorithm in which the pivot element is chosen at random from the subarray being sorted. Random pivot selection is used to reduce the probability of the worst input configuration and make the expected performance better.

Various types of input arrays were used to test the implementation:

- Random arrays
- Already sorted arrays
- Reverse-sorted arrays
- Array with duplicate elements

A deterministic version of Quicksort was also incorporated in the program, which always considered the first element in the list to be the pivot element around which to compare elements.

### Theoretical Analysis

The algorithm used by Quicksort is a divide and conquer algorithm. In each recursive call the array is divided around a pivot so that numbers less than pivot go to one side and numbers greater than pivot go to the other side.

Randomized Quicksort has the following recurrence relation:

$$\mathbf{T(n) = T(k) + T(n - k - 1) + O(n)}$$

where k is number of elements placed in the left partition.

The pivot is also selected uniformly at random, so that each element has the same chance of being selected as pivot. The average partitioning is fairly even, leading to a recursion depth of O(log n).

Indicator random variables can be used to analyze the number of comparisons expected. Define an indicator variable Xij, which will take a value of 1 if elements i and j are compared, and 0 otherwise.

The number of comparisons expected is:

$$\mathbf{E[X] = \Sigma\ E[Xij]}$$

This analysis demonstrates that the number of comparisons expected is proportional to ***nlogn***. So the worst case time of Randomized Quicksort is:

$$\mathbf{O(n \log n)}$$

The worst case time complexity is:

$$\boldsymbol{O(n^2)}$$

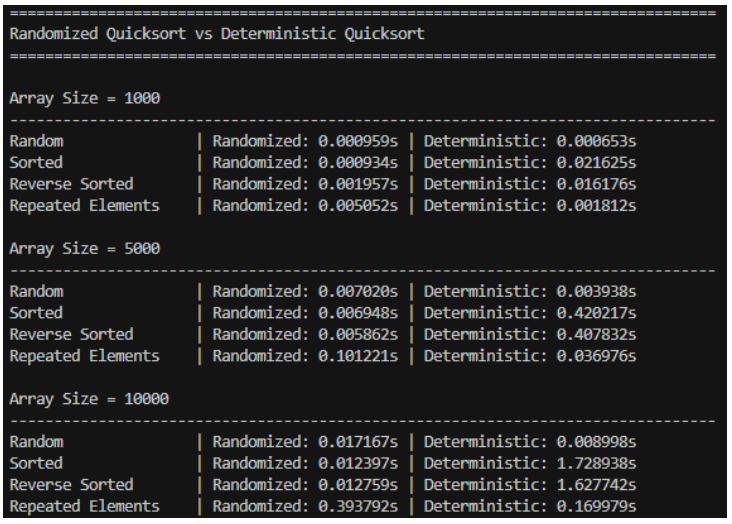
The chances of picking poor pivots, however, are very low.

**Empirical Comparison**

Arrays of sizes 1000, 5000 and 10000 are used to perform the benchmark.

**Figure - 1**

*Result of Code Run*



**Discussion**

The experimental results indicate that a deterministic Quicksort may run slightly faster on random data, as it does not compute random pivots.

Deterministic Quicksort, however, is bad at the sorted and reverse-sorted arrays as they are highly unbalanced. This leads to $O(n^2)$ complexity and to deep recursion, which eventually leads to a recursion overflow.

Randomized Quicksort performed well with all input distributions because its pivot selection is random which means it does not consistently partition badly. This is evidence for the theoretical performance of O(n log n) average case.

The main reasons for the discrepancies between theory and experimentation are the overhead of implementation, the recursion cost in Python, and the randomness in the choice of pivot.

## Part - 2 (Hashing with Chaining)

**Implementation**

The separate chaining hashing was implemented with a hash table. A linked list of key-value pairs is stored in each slot of the hash table. If there are more than one key that maps to the same index, they are added to linked list elements in that bucket.

The implementation can be used for the following operations:

- Insert
- Search
- Delete

Dynamic resizing was also added when the load factor exceeded 0.75.

**Theoretical Analysis**

Let:

n = Number of elements stored in the stack

m = number of buckets

The load factor is: $\boldsymbol{\alpha = n / m}$

Let it be assumed that every key has an equal probability of being inserted into any bucket, this is called simple uniform hashing. Expected operation costs:

- Insert: O(1)
- Search: O(1 + α)
- Delete: O(1 + α)

If the load factor is fixed, these operations can be implemented in O(1) on average.

**Effect of Load Factor**

The load factor is directly related to performance.

Low Load Factor:

- Fewer collisions
- Shorter chains
- Faster searches and deletions

High Load Factor:

- More collisions
- Longer chains
- The time to search and delete files will increase.

The more α increases, the longer the chain is expected to be, which will decrease the efficiency.

**Collision Reduction Strategies**

The load factor is an important aspect of the efficiency of a hash table. It is expressed as the proportion of the number of elements placed in the table to the number of buckets in the table. There will be fewer collisions when the load factor is low since the elements will be spread out across the more buckets. Therefore the chains of each bucket will be short which

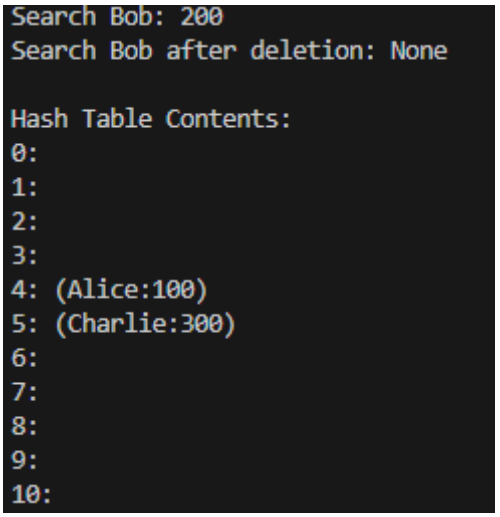
could result in faster searching, inserting and deleting operations. At high load factor, on the other hand, more elements are stuffed into the same buckets, resulting in more collisions. This leads to longer chains, which means that traversing is needed more for searching and deleting, causing a decrease in efficiency. The load factor tends to increase with increase in expected chain length, generally, which has a negative impact on performance.

**Collision Reduction Strategies**

There are several techniques which can be used to reduce the number of collisions and ensure efficient hash table operation. An effective technique is to use a proper hash function which distributes keys as evenly as possible among the buckets, thus minimizing clustering and the chances of collisions. Another approach is dynamic resizing, which means that the size of the hash table grows when the load factor is too high. While resizing, all the elements in the table are rehashed and added to the larger table, thus distributing them more evenly. Also, a relatively low load factor (0.70 to 0.75) reduces chain lengths and helps keep operations efficient. When combined these techniques allow insert, search and delete to still take average expected constant time.

**Figure - 2**

*Output of Hashmap*



## Conclusion

The task highlighted the significance of algorithm design and importance of choosing the appropriate data structures for the task. Randomized Quicksort has an expected running time of O(n log n); on some inputs, deterministic Quicksort will run much slower.

Separate chaining is an efficient implementation of the hash table for insert, search, and delete operations on average. The load factor has a significant impact on performance, and collision management and dynamic resizing are factors to consider.

In general, the theoretical study and experimental testing validate the scalability and efficiency benefits of Randomized Quicksort and Hashing with Chaining.

## References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data structures and algorithms in Python*. John Wiley & Sons.

Kleinberg, J., & Tardos, É. (2006). *Algorithm design*. Pearson.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.

Weiss, M. A. (2013). *Data structures and algorithm analysis in Java* (3rd ed.). Pearson.

Knuth, D. E. (1998). *The art of computer programming, Volume 3: Sorting and searching* (2nd ed.). Addison-Wesley.

Shaffer, C. A. (2013). *A practical introduction to data structures and algorithm analysis* (3rd ed.). Dover Publications.

Miller, B. N., & Ranum, D. L. (2011). *Problem solving with algorithms and data structures using Python*. Franklin, Beedle & Associates.